
Client Discovery Document

AI Relationship Executive Assistant Implementation

Prepared by: Absolv Consulting

Purpose: To align on the business goal, relationship management use case, proposed agentic assistant system, decision loop, technical approach, privacy considerations, and first-phase implementation plan.

Document intent. This draft is designed to support an initial client discovery conversation. The goal is to confirm alignment on **what is being built, why it matters, how the first version should work, and what scope is appropriate for an MVP.**

Executive Summary

This document outlines a proposed implementation for an **AI-powered relationship executive assistant** designed to help manage, prioritize, and nurture high-value relationships across email, LinkedIn, calendar, and, over time, additional systems such as HubSpot, Slack, Zoom transcripts, and manual interaction notes.

The intent of the system is not to create another passive dashboard. Its value comes from turning scattered relationship data into timely, useful action. The assistant should identify which contacts matter most, determine when follow-up is warranted, recommend an appropriate next step, and draft a message for the user to review.

Core objective: The first implementation should demonstrate that the assistant can reliably answer three practical questions: **Who should the user reach out to? Why now? What should they say?**

Business Goal and Alignment

The primary business goal is to create a more structured and intelligent workflow for relationship management. Today, valuable relationship context often lives across email threads, meeting histories, LinkedIn connections, personal memory, and ad hoc notes. The challenge is not simply storing that information, but converting it into useful recommendations at the right moment.

The assistant should reduce the cognitive burden of maintaining a network by surfacing the right contacts at the right time. It should help distinguish between active relationships, dormant relationships, overdue follow-ups, and potential opportunity-driven interactions. At a business level, the intended outcome is stronger relationship maintenance, more meaningful outreach, and more consistent creation of inbound opportunities through the existing network.

This discovery effort is meant to confirm alignment around the desired business outcomes, the right level of automation, the most useful initial data sources, and the amount of user control required before any external action is taken. The purpose is to amplify networking productivity by helping the user consistently nurture existing relationships and create more inbound opportunities from warm network activity.

Relationship Management Use Case

The initial use case is centered on helping the user nurture relationships across an Atlanta-based network, as well as contacts from college, law school, networking groups, business circles, and other professional communities. These relationships already exist across communication channels, but they are difficult to manage manually at scale.

The assistant should create a structured memory around those relationships. It should help the user understand when they last interacted with someone, what was discussed, whether a follow-up is overdue, and whether there is a meaningful reason to reconnect. That turns relationship management from a reactive and memory-based process into a more proactive operating rhythm.

For example, the assistant may identify that a high-value contact was last met with forty-five days ago, that the prior discussion involved a potential introduction or business opportunity, and that no follow-up has been sent. It should then summarize that context, recommend a next step, and draft a short message for review.

Proposed Agentic Assistant System

The proposed solution is a **controlled, human-in-the-loop agentic assistant**. It should not behave like an unconstrained autonomous agent. Instead, it should be composed of defined functions that work together to support the relationship management workflow in a safe, explainable, and measurable way.

At a high level, the system should ingest relevant relationship data, unify contact records across sources, detect meaningful relationship signals, score and prioritize contacts, plan next-best actions, generate recommendations, draft personalized outreach, and update relationship memory based on user actions and feedback.

Core functional components: data ingestion, contact deduplication, profile normalization, signal detection, contact scoring, relationship planning, memory retrieval, message drafting, user review, approval controls, feedback capture, and outcome logging.

The first version should stay focused on the top **100–150 relationships**. This keeps the MVP practical and useful while still proving the value of the system. Over time, the assistant can be expanded to include broader network analysis, introduction opportunities, scheduled monitoring, and additional data sources.

Agent Objective

The agent should have a clear operating objective rather than simply responding to isolated prompts. Its objective is to **maximize timely, relevant relationship actions** while minimizing unnecessary outreach, low-context recommendations, and user review burden.

In practice, this means the assistant should continuously help answer: which relationships need attention, what signal justifies action, what action should happen next, and what context should be used before the user approves anything.

Agentic Behavior and Decision Loop

The assistant is agentic because it does not simply display relationship data or generate one-off messages. It operates through a recurring decision loop: **observe approved data sources, retrieve relevant relationship memory, detect meaningful signals, evaluate priority, plan a next-best action, generate a draft or recommendation, request user approval, record the outcome, and update future scoring.**

Agentic decision loop: observe → retrieve memory → detect signal → score priority → plan action → draft recommendation → request approval → log outcome → improve future decisions.

The system remains human-in-the-loop, but it still performs goal-directed reasoning across multiple steps before presenting a recommendation. The user controls external actions; the assistant handles monitoring, prioritization, planning, drafting, and learning.

Next-Best-Action Space

To make the agent's decision-making concrete, the system should define an action space. Depending on the contact, context, and confidence level, the assistant may recommend a reconnect message, a follow-up after a meeting, a response to an inbound message, an introduction request, an offered introduction, a thank-you note, a scheduled check-in, a reminder, a deferral, a request for more context, or no action.

This action space matters because the assistant is not only ranking contacts. It is deciding what type of relationship action makes sense and whether the timing is strong enough to justify user attention.

Memory Model

The memory layer should be structured so the assistant can improve over time. Contact memory should capture who the person is, how the user knows them, company and role information, relationship category, and relevant personal or professional context. Interaction memory should capture last meetings, recent threads, prior topics, promised follow-ups, and previous drafts or outcomes.

The system should also maintain preference memory and outcome memory. Preference memory captures what kinds of contacts, tones, message styles, and recommendation types the user tends to approve. Outcome memory captures what happened after outreach, such as reply, no response, meeting booked, introduction made, opportunity created, or contact marked as low priority.

Confidence and Uncertainty Handling

Each recommendation should include a confidence level and a short explanation. The assistant should distinguish between high-confidence recommendations, medium-confidence recommendations, contacts that need more context, possible duplicate contacts, weak timing, or insufficient data.

This keeps the agent from overclaiming. It also makes the user review workflow faster because the user can immediately see whether the assistant is making a strong recommendation or asking for clarification.

Technical Architecture Overview

The technical architecture should follow a structured pipeline: **data ingestion** → **normalization** → **signal extraction** → **scoring** → **planning** → **recommendation generation** → **draft creation** → **user approval** → **memory and outcome update**.

Architecture Approach

Ingestion layer. The ingestion layer should support email metadata and relevant thread summaries, calendar history, LinkedIn exports or uploaded contact files, CSV uploads, and manual relationship notes. Email and calendar are especially important in the first version because they provide direct evidence of interaction history, response timing, meeting frequency, and recency.

Normalization layer. The normalization layer should merge fragmented records into unified contact profiles. A single person may appear in email, calendar, LinkedIn, and CSV inputs under different identifiers; the system should reconcile those records into one profile containing name, email, company, title, location, relationship category, last interaction date, meeting history, known context, and saved relationship memory.

Signal and scoring layer. The first version can begin with an explainable weighted scoring model, then evolve into a more personalized machine-learning scoring layer as user feedback, outreach outcomes, and relationship history accumulate. This allows the MVP to remain practical and transparent while still creating a path toward adaptive, user-specific intelligence over time. The system should distinguish useful relationship signals from noise. Meaningful signals may include recent inbound emails, unanswered messages, meetings without follow-up, dormant but high-value contacts, job changes, repeated interactions, or references to opportunities and introductions. Noise may include newsletters, automated messages, receipts, generic calendar events, and low-context activity.

Planning layer. The planning layer converts signals and scores into an actionable recommendation. It determines the desired outcome for a contact, selects the appropriate next-best action, evaluates whether the timing is strong enough, checks whether context is missing, and decides whether to draft a message, create a reminder, defer action, or ask the user for more information.

Drafting and memory layer. The drafting layer should use retrieved relationship context, past interaction summaries, and user writing preferences to produce short, personalized drafts. A retrieval-augmented generation approach is appropriate so that drafts are grounded in the actual relationship history rather than generic AI language.

Tool layer. The agent should interact with a defined set of internal tools rather than free-form system access. These tools may include email lookup, calendar lookup, contact search, memory retrieval, scoring, next-best-action planning, draft generation, approval queue management, reminder creation, audit logging, and approved CRM or contact updates.

Suggested technical stack. A practical initial build would likely use a **Python / FastAPI** backend, **PostgreSQL** for structured relationship data, **background jobs** for ingestion and scoring, a **vector or embedding-backed memory layer** for retrieval, an **orchestration**

layer for routing tasks between agent functions, and a **React or Next.js dashboard** for user review. Initial integrations may include **Gmail or Outlook, Google Calendar or Microsoft Calendar, LinkedIn export uploads, and CSV ingestion.**

User Review and Approval Workflow

The first implementation should remain fully human-in-the-loop. The assistant may monitor approved sources, update internal scores, generate recommendations, and draft messages, but the user should review, edit, approve, reject, or defer each recommendation before anything is sent.

Each recommendation should present the contact, the reason the person is being surfaced, the supporting relationship context, the suggested action, the confidence level, and the draft message. The review workflow should make it easy for the user to quickly approve a draft, revise it, reject it, defer it, or mark it as not relevant.

Approval Boundaries

Without explicit approval, the system may scan approved data sources, update internal scores, refresh the recommendation queue, retrieve memory, generate drafts, flag missing context, and prepare suggested actions. With explicit approval, it may send a message, create a reminder, update a CRM or contact record, mark an outreach as complete, or schedule a follow-up task.

MVP control rule: No outbound communication should be sent automatically in the first version. The assistant should **prepare and recommend**; the user should **decide and approve**.

Feedback and Learning Signals

The approval workflow should create a clear feedback loop. An approved recommendation means the contact, timing, and action were useful. A heavily edited draft means the action may have been correct, but the tone, wording, or context needs improvement. A rejection indicates that the recommendation was not useful. A deferral means the contact may matter, but the timing was weak. A “not relevant” label means the scoring model overvalued that contact, signal, or relationship category.

These feedback signals should be logged and used to improve future scoring, message style, timing, and next-best-action selection.

Expected Workflow Impact

The expected impact is a meaningful reduction in the manual work required to manage high-value relationships. Instead of searching through inboxes, calendars, LinkedIn, and memory to figure out who needs attention, the user would review a prioritized queue of recommended actions.

The assistant should help prevent valuable relationships from going cold, improve follow-up consistency, and reduce the time spent drafting tailored messages. It should also make it easier to act on opportunity signals that might otherwise be missed, such as reconnecting after a meeting, following up after an introduction, or checking in when a valuable contact has gone dormant.

In that sense, the benefit is not simply convenience. The system is designed to turn relationship management into a more structured, proactive workflow that supports consistent network maintenance and better opportunity creation.

Agent performance metrics: recommendation approval rate, rejection rate, deferral rate, average draft edit distance, reply rate, meetings booked, introductions created, dormant contacts revived, false-positive recommendations, high-value contacts reviewed, and estimated time saved per week.

Phased Rollout Plan

Phase 1	Discovery and alignment. Confirm the business objective, highest-value relationship categories, initial data sources, privacy expectations, agent objective, action space, and definition of a successful recommendation. Deliverables should include a workflow map, MVP scope, data source list, scoring criteria, privacy approach, and agent operating rules.
Phase 2	MVP backend build. Set up the database, ingest CSV or LinkedIn-exported contacts, connect email and calendar metadata where appropriate, create unified contact profiles, implement the first scoring model, and define memory structures.
Phase 3	Agentic workflow and dashboard. Build the ranked contact list, contact profile views, recommendation explanations, planning layer, confidence labels, draft generation flow, and the approve/edit/reject/defer workflow.
Phase 4	Pilot and refinement. Test the system with real relationship data and tune scoring, next-best-action selection, confidence thresholds, and message quality based on approval rate, rejection rate, draft edits, and actual relationship outcomes.
Phase 5	Expanded intelligence layer. Add scheduled monitoring, HubSpot, Slack, Zoom transcripts, CRM records, meeting notes, in-person interaction logging, and more advanced learning from outcomes.

Data Privacy and Access Considerations

Because the system touches sensitive personal and professional communication data, privacy should be treated as a core design requirement. The platform should ingest and retain only the data necessary to support the use case. In many scenarios, it may not be necessary to permanently store full email bodies; storing metadata, timestamps, interaction summaries, relationship signals, and user-approved memory notes may be sufficient.

Access to email, calendar, and related systems should rely on secure OAuth-based authentication where possible. Tokens should be stored securely, sensitive data should be encrypted, and the application should maintain clear separation between user data and system logic. The user should also be able to delete imported contacts, stored summaries, memory notes, and account-level data.

The system should support scheduled scans of approved sources to detect new relationship signals, but those scans should only update internal recommendations and should not trigger external communication without approval.

The system should maintain an audit trail showing what data was accessed, what recommendation was generated, what draft was produced, what confidence level was assigned, and what action the user took. That transparency is important both for trust and for operational control.

Implementation Estimate and Pricing

The work can be structured as a phased engagement. A sensible first engagement would focus on discovery, architecture finalization, MVP workflow design, and the first working version of the assistant.

For an initial MVP, the estimated implementation window is approximately **four to six weeks**, depending on integration complexity and the amount of dashboard refinement required. A lighter version based on uploaded contacts plus email/calendar metadata will move faster than a version requiring deeper API integrations, more robust data pipelines, or multi-source synchronization. The overall integration and implementation is expected to take around **ten to twelve weeks**.

Recommended starting structure. We believe that a foundational implementation engagement would be approximately **\$12,500–\$15,000**. This would typically include discovery, workflow mapping, technical architecture, core ingestion, contact normalization, initial scoring logic, agentic decision-loop design, next-best-action planning, recommendation generation, draft creation, confidence labeling, outcome logging, and the user review-and-approval workflow. After the MVP is delivered, ongoing maintenance, model tuning, infrastructure support, and future feature development can be scoped separately based on usage, integration complexity, and the pace of new feature requests. This can be structured as a monthly optimization retainer or as fixed-price implementation sprints for additional integrations and product enhancements.

Key Alignment Questions

Before implementation begins, several questions should be answered to confirm scope and avoid misalignment.

1. Which relationship categories matter most for the first version: Atlanta network, college, law school, professional contacts, investors, potential clients, or another group?
2. Which data sources should be included in the MVP, and which should be deferred until later?
3. Should the first version only draft messages, or should it also support approved sending within the platform?
4. What should count as a successful recommendation: a reminder, a sent message, a reply, a booked meeting, an introduction, or a business opportunity?
5. How much historical email and calendar data should be included in the initial import?
6. Should the system retain full communication history, or only metadata, summaries, and relationship memory?
7. What level of dashboard functionality is needed in the MVP versus later phases?
8. Which relationship signals should carry the greatest scoring weight initially?
9. What actions should the assistant be allowed to recommend in the MVP?

10. What confidence threshold should be required before surfacing a recommendation?
11. Which feedback signals should be used to tune scoring and writing style?
12. What privacy controls, deletion rights, and audit-log requirements must be in place before live use?

Next Steps

The next step is to confirm alignment on the business goal, the MVP scope, and the initial data sources. Once that is agreed, the project can move into technical design, schema definition, integration planning, scoring-model design, action-space definition, memory modeling, and approval-boundary design.

The first build should remain narrow but useful: ingest contact and interaction data, create unified profiles, rank the top **100–150 relationships**, identify meaningful follow-up opportunities, plan next-best actions, generate personalized draft messages, assign confidence levels, log outcomes, and require user approval before any outbound action.

That scope should be sufficient to determine whether the assistant can provide real relationship intelligence and materially reduce the manual burden of staying connected with important people.